Assignment - 11.3
Name:A.Vignesh
Roll no:2303A51640
Batch:09

Lab 11: Data Structures with AI Implementing Fundamental Data Structures using AI Assistance

Lab Objectives:

By the end of this lab, students will be able to:

Week6 -

Wednesday

• Design and implement fundamental data structures in Python using AI assistance.

• Effectively prompt AI tools (e.g., GitHub Copilot) for code generation, optimization, and documentation.

• Understand and compare core data structures: Arrays, Linked Lists, Stacks, Queues, Priority Queues, Trees, and Graphs.

• Improve code readability, efficiency, and maintainability using AI-generated suggestions.

Learning Outcomes

After completing this lab, students will be able to:

• Apply appropriate data structures to solve real-world problems.

• Analyze time and space complexity of different data structure operations.

• Use AI tools responsibly to assist (not replace) logical thinking and problem-solving.

• Validate, test, and refine AI-generated code.

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:
   o Add a contact
   o Search for a contact
   o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:
   o Insertion efficiency
   o Deletion efficiency

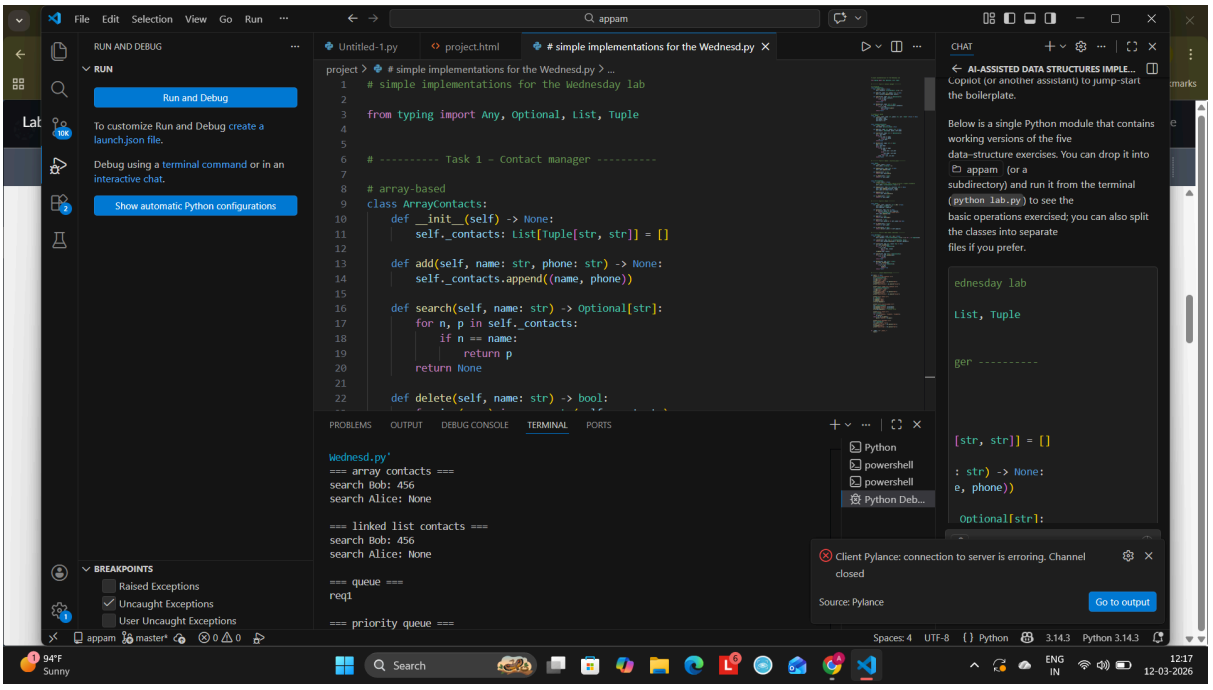
Expected Outcome

• Two working implementations (array-based and linked-list-based).
• A brief comparison explaining performance differences.

prompt: Smart Contact Manager (Arrays & Linked Lists)

Scenario

code:



Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:

o enqueue() method

o dequeue() method

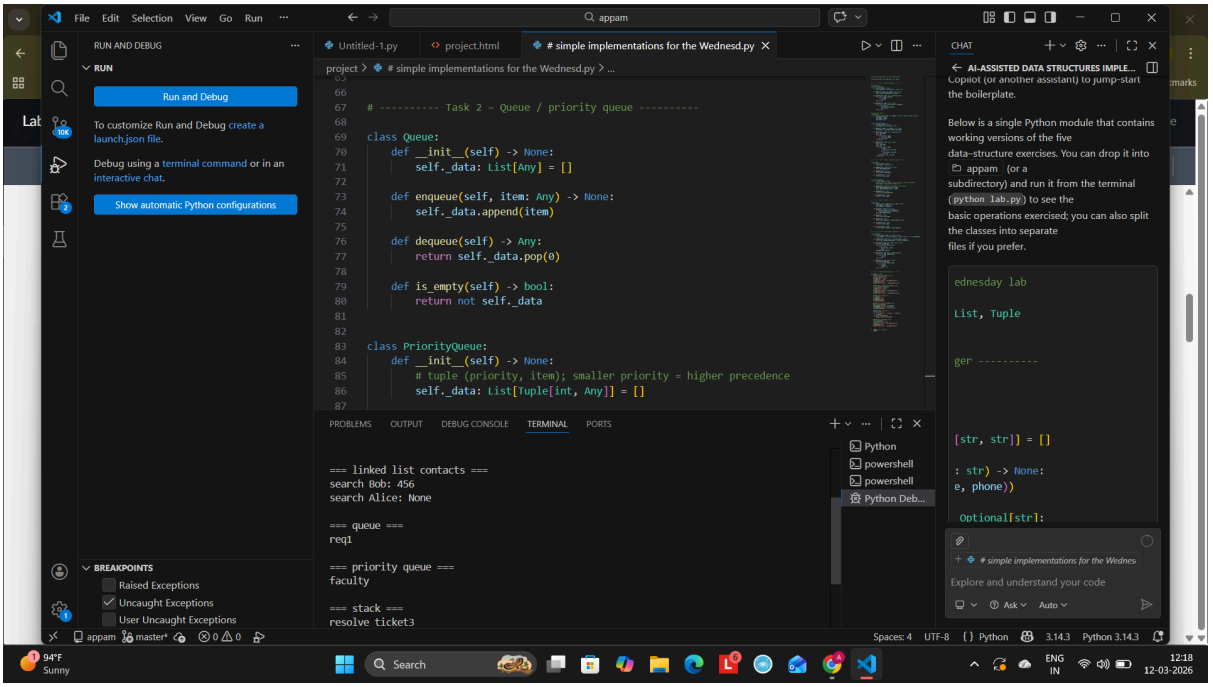
4. Test the system with a mix of student and faculty requests.

Expected Outcome

• Working queue and priority queue implementations.
• Correct prioritization of faculty requests.

prompt:Library Book Search System (Queues & Priority Queues)

Code:



Task 3: mergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.
2. Provide the following operations:

o push(ticket)

o pop()

o peek()

3. Simulate at least five tickets being raised and resolved.
4. Use GitHub Copilot to suggest additional stack operations such as:

o Checking whether the stack is empty

o Checking whether the stack is full (if applicable)

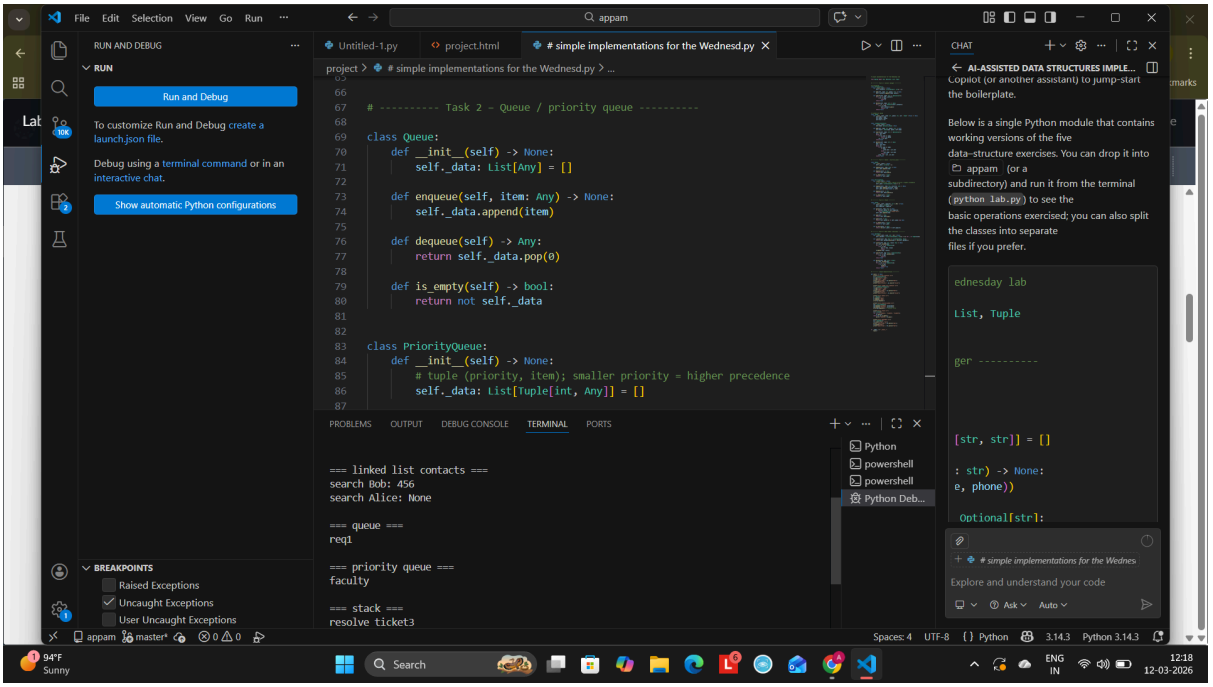
Expected Outcome

• Functional stack-based ticket management system.

• Clear demonstration of LIFO behavior.

prompt:mergency Help Desk (Stack Implementation)

Code:



Task 4: Hash Table

Object

Use AI to generate a hash table with:

- Insert
- Search
- Delete

Starter Code

class HashTable:

pass

Expected Outcome

- Collision handling using chaining
- Well-commented methods

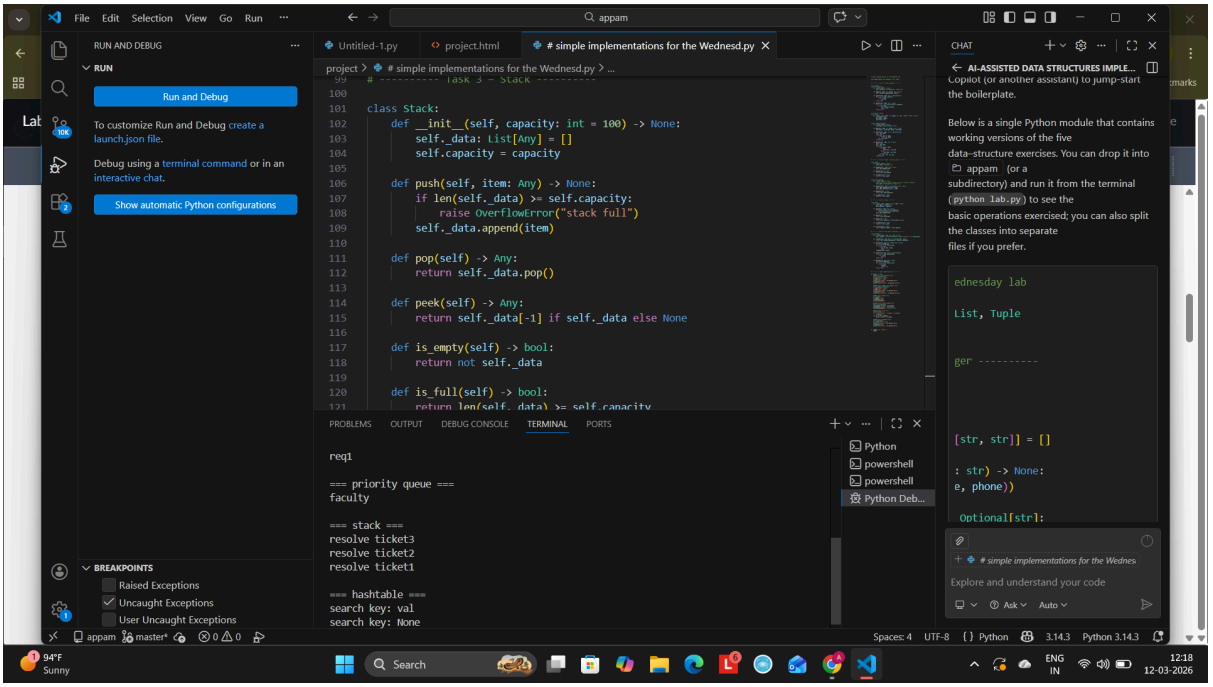
prompt:Hash Table

Object

Use AI to generate a hash table with:

- Insert

Code:



Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

•

prompt:Real-Time Application Challenge

Scenario

Code:

File Edit Selection View Go Run appam

RUN AND DEBUG

RUN

Run and Debug

To customize Run and Debug create a launch.json file.

Debug using a terminal command or in an interactive chat.

Show automatic Python configurations

BREAKPOINTS

- Raised Exceptions
- Uncaught Exceptions
- User Uncaught Exceptions

Untitled-1.py | project.html | # simple implementations for the Wednesd.py

project > # simple implementations for the Wednesd.py > ...

```python
# ---------- Task 4 - Hash table (chaining) ----------

class HashTable:
    def __init__(self, size: int = 16) -> None:
        self._buckets: List[List[Tuple[Any, Any]]] = [[] for _ in range(size)]

    def _bucket(self, key: Any) -> List[Tuple[Any, Any]]:
        return self._buckets[hash(key) % len(self._buckets)]

    def insert(self, key: Any, value: Any) -> None:
        b = self._bucket(key)
        for i, (k, _) in enumerate(b):
            if k == key:
                b[i] = (key, value)
                return
        b.append((key, value))

    def search(self, key: Any) -> Optional[Any]:
        for k, v in self._bucket(key):
            if k == key:
                return v
        return None
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python
powershell
powershell
Python Deb...

```
=== priority queue ===
faculty

=== stack ===
resolve ticket3
resolve ticket2
resolve ticket1

=== hashtable ===
search key: val
search key: None
PS C:\Users\appam>
```

CHAT

AI-ASSISTED DATA STRUCTURES IMPLE...

Copilot (or another assistant) to jump-start the boilerplate.

Below is a single Python module that contains working versions of the five data-structure exercises. You can drop it into appam (or a subdirectory) and run it from the terminal (`python lab.py`) to see the basic operations exercised; you can also split the classes into separate files if you prefer.

```
ednesday lab

List, Tuple


ger ----------



[str, str]] = []

: str) -> None:
e, phone))

Optional[str]:
```

simple implementations for the Wednes

Explore and understand your code

Ask Auto

appam master* 0 0 0

Spaces: 4 UTF-8 { } Python 3.14.3 Python 3.14.3

94°F Sunny Search ENG IN 12:19 12-03-2026